

S I G A A

INFORME TÉCNICO

Tecnologías que utilizaremos en el proyecto SIGAA

Línea base tecnológica para el desarrollo, pruebas, despliegue y operación segura del Sistema Integral de Gestión Académica y Alertas para establecimientos escolares.

PROYECTO	SIGAA · Capstone PTY4614
ENFOQUE	Arquitectura web modular, portable y segura
DOCUMENTO	Propuesta redactada en futuro para aprobación del equipo
FECHA	26 de agosto de 2026

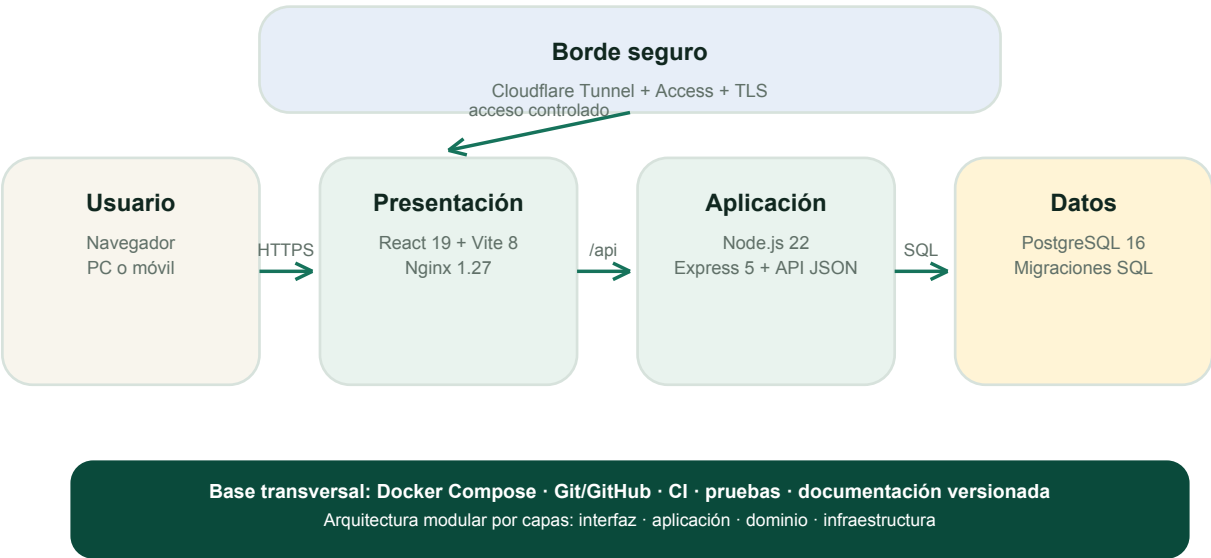
DECISIÓN PROPUESTA

Adoptaremos una base simple y mantenible: React + Node.js/Express + PostgreSQL, empaquetada con Docker y operada mediante Nginx, Cloudflare y automatización en GitHub.

01 · VISIÓN GENERAL

Resumen ejecutivo

Usaremos tecnologías ampliamente adoptadas, compatibles entre sí y adecuadas para que un equipo de tres integrantes pueda desarrollar, probar y desplegar SIGAA sin depender de configuraciones particulares de cada computador.



Experiencia web Usaremos React 19 con Vite 8 para construir interfaces responsivas por rol. La aplicación será servida por Nginx.	Lógica de negocio Usaremos Node.js 22 y Express 5 para exponer una API JSON y organizar el dominio académico en módulos.
Persistencia Usaremos PostgreSQL 16 por su integridad relacional, transacciones y capacidad para representar cursos, evaluaciones, asistencia y alertas.	Entrega segura Usaremos Docker Compose, GitHub Actions, Cloudflare Tunnel/Access y redes aisladas para una operación reproducible.

Criterios de selección

- Portabilidad entre macOS, Windows y Linux mediante contenedores.
- Curva de aprendizaje coherente: JavaScript/TypeScript en cliente y servidor.
- Seguridad por capas, con exposición mínima de servicios y trazabilidad de acciones.
- Capacidad de crecer desde un prototipo hasta módulos productivos sin rediseñar toda la solución.
- Costos controlados gracias a componentes de código abierto y servicios con planes adecuados al proyecto académico.

02 · CAPA DE PRESENTACIÓN

Frontend y experiencia de usuario

Construiremos una aplicación web adaptable a escritorio y teléfono. La misma base atenderá a docentes, estudiantes y futuros perfiles, mostrando únicamente las funciones autorizadas para cada rol.

Tecnología	Versión base	Uso que le daremos	Motivo
React	19.2.8	Componentes, estado de interfaz, vistas por rol y formularios.	Favorecerá reutilización, pruebas y evolución incremental.
Vite	8.2.2	Servidor de desarrollo y compilación optimizada del frontend.	Entregará ciclos rápidos y una configuración ligera.
TypeScript	Migración gradual	Tipado de contratos, modelos y módulos productivos.	Reducirá errores al crecer el código sin frenar el prototipo actual.
CSS responsivo	Estándar web	Diseño premium, accesibilidad, navegación móvil y adaptación visual.	Evitará depender de una biblioteca visual cerrada.
Nginx	1.27-alpine	Servirá archivos compilados, resolverá rutas SPA y redirigirá /api.	Será un borde web eficiente y configurable.

Responsabilidades del frontend

- Presentará navegación, formularios, tablas, paneles y retroalimentación al usuario.
- Consumirá la API exclusivamente mediante contratos JSON; no se conectará directamente a PostgreSQL.
- Aplicará estados de carga, mensajes de error y validaciones de experiencia, sin reemplazar la validación del servidor.
- Respetará navegación por teclado, contraste legible y diseño mobile-first en las pantallas críticas.

Decisión de evolución

El prototipo existente conservará React. Antes de implementar módulos productivos, migraremos gradualmente los componentes y contratos críticos a TypeScript. Esta transición deberá quedar registrada y probada por módulo.

03 · SERVICIOS DE APLICACIÓN

Backend, API y arquitectura

Implementaremos inicialmente un monolito modular. Esta forma de trabajo mantendrá el despliegue simple y, al mismo tiempo, separará las responsabilidades internas para que el sistema pueda crecer.

Node.js 22

Usaremos el runtime LTS definido por el proyecto y módulos ESM. Compartiremos conocimientos y tipos con el frontend.

Express 5

Construiremos endpoints HTTP/JSON, middleware, validación, autorización y manejo centralizado de errores.

pg 8.23

Accederemos a PostgreSQL con consultas parametrizadas, transacciones y control explícito del SQL.

Organización por capas

Capa	Responsabilidad que asumirá	Ejemplos SIGAA
Interfaz	Recibirá solicitudes, validará formato y traducirá respuestas HTTP.	Rutas, controladores, middleware.
Aplicación	Coordinará casos de uso y políticas de autorización.	Pasar asistencia, registrar nota, crear clase.
Dominio	Representará reglas académicas sin depender de infraestructura.	Curso, matrícula, evaluación, anotación, alerta.
Infraestructura	Conectará almacenamiento, auditoría y servicios externos.	PostgreSQL, repositorios, notificaciones.

El motor de alertas, la auditoría y las notificaciones comenzarán como módulos internos. Solo los separaremos en servicios independientes si la carga, la autonomía de despliegue o el aislamiento operativo justifican ese costo.

Tecnología que reevaluaremos

NestJS no será requisito inicial. Lo evaluaremos si aumenta la complejidad de módulos, inyección de dependencias y contratos. Cualquier migración requerirá una ADR y no deberá interrumpir la entrega funcional.

04 · PERSISTENCIA

Base de datos y gestión de información

Usaremos PostgreSQL porque el dominio escolar es principalmente relacional y requiere consistencia: una nota pertenece a una evaluación y estudiante; una asistencia pertenece a una clase, matrícula y fecha.

PostgreSQL 16-alpine

Será la base transaccional principal. Aplicaremos claves foráneas, restricciones, índices y transacciones para proteger la integridad académica.

Migraciones SQL versionadas

Cada cambio de esquema viajará junto al código, tendrá orden reproducible y podrá revisarse mediante Git antes de llegar al servidor.

Prácticas que aplicaremos

- Modelaremos usuarios, roles, docentes, estudiantes, apoderados, cursos, asignaturas, matrículas, clases, asistencia, evaluaciones, calificaciones, anotaciones, alertas y auditoría.
- Usaremos consultas parametrizadas y evitaremos concatenar datos del usuario en SQL.
- Crearemos índices a partir de consultas reales y mediremos como criterio inicial las consultas principales con datos sintéticos.
- Mantendremos datos de desarrollo anónimos o sintéticos; no versionaremos información personal real.
- Definiremos política de respaldo, restauración y retención antes de cualquier uso productivo.

Por qué no usaremos NoSQL como base principal

NoSQL no aportará una ventaja clara al núcleo actual. La consistencia entre matrículas, evaluaciones, calificaciones y asistencia es más importante que la flexibilidad de documentos. Podremos incorporar otro almacenamiento en el futuro para un caso específico, pero no sustituirá la fuente transaccional sin evidencia técnica.

Meta inicial de rendimiento

Tomaremos como referencia que las consultas principales respondan bajo 500 ms con datos sintéticos en un entorno controlado. Esta cifra será un criterio inicial, no una garantía contractual, y se ajustará después de medir el volumen real.

05 · EJECUCIÓN Y DESPLIEGUE

Docker, Nginx y topología operativa

Empaquetaremos cada componente para que todos los integrantes ejecuten la misma versión del sistema. La configuración local y la del servidor compartirán imágenes, variables y verificaciones de salud.



Servicio	Imagen/base	Función	Exposición prevista
web	nginx:1.27-alpine	Frontend compilado y proxy /api.	Local: loopback. Servidor: solo red Docker.
api	node:22-alpine	Casos de uso y API JSON.	Local: loopback. Servidor: solo red interna.
db	postgres:16-alpine	Persistencia transaccional y volumen.	Nunca se publicará a Internet.
cloudflared	cloudflare/cloudflared	Túnel saliente hacia Cloudflare.	Sin puertos entrantes en el host.

Usaremos healthchecks para web, API y base de datos; volúmenes persistentes para PostgreSQL; variables de entorno fuera de Git; y redes separadas de frontend/backend. Docker Compose será el contrato de ejecución del equipo y del servidor.

Aislamiento

En el servidor no publicaremos directamente web, API ni base de datos. El contenedor de Cloudflare iniciará una conexión saliente y será el único puente controlado hacia la aplicación.

06 · SEGURIDAD Y PRIVACIDAD

Acceso controlado y defensa por capas

Aplicaremos mínimo privilegio y denegación por defecto. Los integrantes podrán usar la aplicación sin recibir acceso general al computador servidor ni credenciales directas de la base de datos.

Control	Tecnología o práctica	Resultado esperado
Transporte	HTTPS/TLS mediante Cloudflare	Cifrado del tráfico entre usuario y borde.
Acceso externo	Cloudflare Tunnel + Access	Identidad permitida antes de llegar a SIGAA; sin puertos entrantes.
Autorización	RBAC en la API	Docente, estudiante y otros roles solo ejecutarán acciones autorizadas.
Sesiones	Sesiones revocables seguras o tokens de corta duración	Estrategia final pendiente de ratificación técnica.
Credenciales	Hash resistente y secretos fuera de Git	No almacenaremos contraseñas en texto ni compartiremos claves del servidor.
Aplicación	Validación, SQL parametrizado y auditoría	Menor superficie de inyección y trazabilidad de cambios sensibles.
Navegador	CSP, X-Content-Type-Options, X-Frame-Options, Referrer/Permissions Policy	Reducción de riesgos comunes en el cliente.

Decisión pendiente que no ocultaremos

Antes de producción definiremos formalmente el mecanismo de autenticación, la librería y parámetros de hash, expiración/renovación, recuperación de cuenta, MFA si corresponde y política de auditoría. La línea base exige el resultado de seguridad, pero aún no fija una biblioteca concreta.

Datos personales

Durante el desarrollo utilizaremos registros sintéticos. Antes de operar con datos reales estableceremos perfiles de acceso, retención, respaldo, restauración, registro de actividad y procedimiento de incidentes de acuerdo con la política del establecimiento y la normativa aplicable.

07 · CALIDAD Y AUTOMATIZACIÓN

Pruebas, integración continua y observabilidad

Cada cambio deberá ser verificable. Automatizaremos las comprobaciones repetibles y conservaremos evidencia para que el equipo pueda revisar qué se construyó, quién lo cambió y si la versión es desplegable.

1. Código	2. Dependencias	3. Pruebas	4. Build	5. Docker	6. Evidencia
Branch + commit	npm ci	Unitarias e integración	Vite web	config + up --wait	logs + resultado CI

Git y GitHub

Usaremos ramas, commits claros, pull requests y revisión. GitHub será la fuente de código y evidencias versionadas.

GitHub Actions

Ejecutaremos Node 22, pruebas de API, build web y levantamiento de Docker Compose con healthchecks.

Pruebas

Comenzaremos con el runner nativo de Node para unitarias e integración; agregaremos E2E a flujos críticos.

Estrategia de validación

- Unitarias: reglas de dominio, validadores y servicios sin dependencias externas.
- Integración: API, repositorios, migraciones y transacciones contra PostgreSQL controlado.
- End-to-end: inicio de sesión, asistencia, clases futuras, notas, anotaciones y consultas del estudiante.
- Seguridad: autorización por rol, entradas maliciosas, manejo de sesión y exposición de servicios.
- Rendimiento: consultas y endpoints principales con volumen sintético representativo.
- Verificación visual: escritorio y teléfono en navegadores actuales.

Definition of Done técnica

Una historia no se considerará terminada solo porque funcione en el computador del autor. Deberá incluir código versionado, pruebas pertinentes, documentación actualizada, revisión y ejecución satisfactoria en Docker/CI.

08 · DOCUMENTACIÓN Y TRABAJO COLABORATIVO

Tecnologías para mantener conocimiento y trazabilidad

La documentación será parte del producto. La escribiremos junto con los cambios para reducir dependencia de la memoria del equipo y facilitar las evaluaciones de Capstone.

Herramienta	Uso que le daremos	Artefactos
Markdown	Documentación viva y revisable en Git.	README, manuales, bitácora, actas, criterios.
Mermaid / PlantUML	Diagramas mantenibles como texto versionado.	Arquitectura, secuencias, estados, modelo ER.
ADR	Registro de decisiones y sus consecuencias.	Stack, Docker, seguridad, cambios relevantes.
Scrum	Planificación y control iterativo.	Backlogs, tablero, Daily, Review, Retrospective.
PDF	Entregables estables para evaluación y aprobación.	Informes, Gantt, manuales y cierres.

Reglas que seguiremos

- Toda decisión material tendrá fecha, estado y responsable propuesto.
- Los archivos editables y sus versiones finales conservarán nombres estables y ubicación predecible.
- Los documentos distinguirán hechos verificados, propuestas y decisiones pendientes.
- Cada sprint asociará historias, commits, pruebas, capturas, Review y Retrospective.
- El repositorio no contendrá contraseñas, tokens, llaves privadas ni datos personales reales.

Objetivo colaborativo

Los tres integrantes podrán desarrollar, probar y documentar con la misma base tecnológica. Las tareas se distribuirán equitativamente, mientras las decisiones de equipo se aprobarán y registrarán de forma conjunta.

09 · DECISIONES Y ALTERNATIVAS

Por qué elegiremos esta línea base

La selección busca maximizar el valor del proyecto con un equipo pequeño. No afirmamos que exista una tecnología universalmente mejor; elegiremos la combinación que reduzca complejidad accidental y mantenga opciones de evolución.

Alternativa	Ventaja	Razón para no adoptarla ahora
Django + PostgreSQL	ORM y administración maduros.	Añadiría Python a una base que busca compartir JavaScript/TypeScript entre frontend y backend.
Spring Boot	Ecosistema empresarial robusto.	Tendría mayor costo inicial de estructura y aprendizaje para el alcance del prototipo.
Supabase	Acelera autenticación y CRUD.	Aumentaría dependencia externa y reduciría el control del backend y de la topología privada.
NestJS	Estructura y DI más prescriptivas.	Lo reservaremos para cuando la complejidad demuestre que Express deja de ser suficiente.
Microservicios	Despliegue independiente por servicio.	Serían complejidad operativa prematura para tres integrantes y el volumen actual.
NoSQL principal	Esquemas flexibles.	El núcleo escolar necesita relaciones, restricciones y transacciones fuertes.

Principios de evolución

- Mediremos antes de escalar o reemplazar componentes.
- Registraremos cambios tecnológicos en ADR con contexto, decisión y consecuencias.
- Evitaremos migraciones simultáneas que no aporten una mejora verificable al usuario.
- Priorizaremos compatibilidad, seguridad y mantenibilidad sobre novedad tecnológica.

Estado de la decisión

La línea base está aceptada técnicamente para el prototipo y requiere ratificación formal del equipo. El informe propone su adopción para las siguientes iteraciones; no convierte automáticamente las decisiones pendientes en acuerdos cerrados.

10 · PLAN DE ADOPCIÓN

Cómo incorporaremos las tecnologías

Aplicaremos la línea base por etapas para que cada sprint produzca evidencia funcional y reduzca riesgos antes de trabajar con información real.

Evidencia de término		
Etapas	Aplicación tecnológica	Evidencia de término
1 · Base reproducible	Docker Compose, Node 22, React/Vite, PostgreSQL 16 y Nginx.	Entorno levanta con healthchecks en cada equipo.
2 · Contratos y dominio	API JSON, capas, migraciones y módulos docente/estudiante.	Flujos críticos integrados con datos sintéticos.
3 · Seguridad	RBAC, autenticación ratificada, auditoría, secretos y headers.	Pruebas de rol y revisión de exposición aprobadas.
4 · Calidad	CI, unitarias, integración, E2E y mediciones iniciales.	Pipeline verde y Definition of Done cumplida.
5 · Operación	Cloudflare Tunnel/Access, respaldo y restauración.	Despliegue seguro verificable y recuperación ensayada.
6 · Validación	Observabilidad, feedback y ajustes por uso real.	Resultados de prueba y decisiones actualizadas.

Decisiones que cerraremos antes de producción

- Ratificación del stack por los tres integrantes y asignación de responsables técnicos.
- Mecanismo de autenticación, ciclo de sesión, recuperación de cuenta y hash de contraseñas.
- Política de respaldo, restauración, retención, auditoría e incidentes.
- Objetivos de disponibilidad y rendimiento basados en volumen real.
- Dominio, cuentas de Cloudflare y política de acceso del establecimiento.

Resultado esperado

Al finalizar la adopción, cualquier integrante autorizado podrá clonar el repositorio, ejecutar SIGAA con Docker, desarrollar un cambio, validarlo mediante pruebas y CI, y desplegarlo sin conocer credenciales de la base de datos ni obtener acceso general al servidor.

11 · CONCLUSIÓN

Línea base tecnológica propuesta

Usaremos una arquitectura web moderna y deliberadamente sobria. La combinación seleccionada permitirá avanzar rápido durante Capstone sin renunciar a integridad de datos, seguridad, pruebas ni trazabilidad.

Área	Tecnología que usaremos	Compromiso principal
Frontend	React 19 + Vite 8 + migración gradual a TypeScript	Interfaz responsiva, reutilizable y accesible.
Backend	Node.js 22 + Express 5 + pg 8	API JSON modular y reglas centralizadas.
Datos	PostgreSQL 16 + migraciones SQL	Consistencia, transacciones y esquema versionado.
Infraestructura	Docker Compose + Nginx	Portabilidad, aislamiento y ejecución uniforme.
Acceso	Cloudflare Tunnel + Access + TLS	Entrada cifrada sin publicar servicios internos.
Entrega	GitHub + GitHub Actions	Colaboración, revisión y validación automática.
Calidad	Pruebas unitarias, integración, E2E, seguridad y rendimiento	Evidencia antes de declarar un incremento terminado.
Documentación	Markdown + Mermaid/PlantUML + ADR + PDF	Conocimiento y decisiones trazables.

Recomendación

Aprobar esta línea base para el desarrollo de SIGAA y registrar la ratificación del equipo en ADR-001. Las decisiones pendientes de autenticación, respaldo y operación deberán cerrarse antes de utilizar datos reales o declarar un entorno productivo.

ALCANCE DEL INFORME

Este documento expresa lo que el equipo propone usar. Las versiones corresponden a la configuración actual del repositorio al 26 de agosto de 2026. Toda actualización relevante deberá registrarse mediante control de versiones y una decisión técnica trazable.

SIGAA

Tecnología con propósito: una plataforma escolar segura, mantenible y verificable.